



BEN-GURION UNIVERSITY OF THE NEGEV
Faculty of Engineering Sciences
Department of Software and Information Systems Engineering

Deep Reinforcement Learning

Assignment #2

Due date: 24/12/2025

Lecturer: Dr. Gilad Katz

Students: Mark Feldman (320827637), Tamir Levin (315765347)

Section 1 - Monte-Carlo Policy Gradient (REINFORCE)

Question 1: What does the value of the advantage estimate reflect? Why is it better to follow the gradient computed with the advantage estimate instead of just the return itself?

In policy gradient methods we often use an advantage estimate of the form $A_t = R_t - b(s_t)$ where $b(s_t)$ is usually the value function V_π . The value of the advantage A_t measures how much better or worse the chosen action a_t was compared to what the policy expected from state s_t :

- $A_t > 0$: the obtained return is higher than the baseline $\Rightarrow$ the action was better than average in that state.
- $A_t < 0$: the return is lower than the baseline $\Rightarrow$ the action was worse than average.
- $A_t \approx 0$: the action is about as good as expected.

Instead of using the raw return R_t in the policy gradient update, using A_t subtracts a baseline that does not depend on the action. This does not change the expected gradient, but it reduces its variance as the learning signal is centered around zero and becomes less noisy. As a result, the updates are more stable and the policy can converge faster than when using R_t alone.

Question 2: The reduction of the baseline does not introduce bias to the expectation of the gradient since:

$$E_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a_t | s_t) b(s_t)] = 0$$

What is the prerequisite condition for that equation to be true? Prove the equation.

First, we assume that the baseline $b(s_t)$ does not depend on the action a_t . It can depend on the state s_t , but it must be independent of which action is taken so that $b = b(s_t)$. Next, we want to show that:

$$E_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a_t | s_t) b(s_t)] = 0$$

Since $b(s_t)$ does not depend on the action, we can treat it as a constant inside the expectation over a_t :

$$E_{a_t}[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) b(s_t)] = b(s_t) E_{a_t}[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)] = b(s_t) \sum_a \pi_{\theta}(a | s_t) \nabla_{\theta} \log \pi_{\theta}(a | s_t)$$

To simplify this, we use the identity:

$$\nabla \log x = \frac{\nabla x}{x}$$

So it can be written as:

$$b(s_t) \sum_a \pi_{\theta}(a | s_t) \nabla_{\theta} \log \pi_{\theta}(a | s_t) \implies b(s_t) \sum_a \nabla_{\theta} \pi_{\theta}(a | s_t)$$

Applying the linearity of gradients it can be written as:

$$b(s_t) \sum_a \nabla_{\theta} \pi_{\theta}(a | s_t) = b(s_t) \nabla_{\theta} \sum_a \pi_{\theta}(a | s_t)$$

Because $\pi_{\theta}(a | s_t)$ is a probability distribution,

$$\sum_a \pi_{\theta}(a | s_t) = 1$$

Taking the gradient of a constant gives:

$$\nabla_{\theta} 1 = 0$$

Therefore,

$$E_{a_t}[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)] = 0$$

and so,

$$E_{\pi_{\theta}}[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) b(s_t)] = E_{s_t}[b(s_t) \cdot 0] = 0$$

REINFORCE Algorithms Implementation

We implemented two variants of the REINFORCE algorithm and evaluated them on the `CartPole-v1` environment: the basic REINFORCE algorithm that uses Monte Carlo returns directly for policy gradient updates, and a version that uses a learned value function baseline to reduce variance.

Experimental Setup

Both algorithms were trained for up to 2000 episodes using a discount factor $\gamma = 0.99$. The policy network consists of a two-layer feedforward network with 256 hidden layers. For the baseline version, a separate value network with identical architecture approximates the state-value function. We consider the environment solved when the average reward over the last 100 episodes exceeds 475.

Results

Figure 1 shows the learning curve for the basic REINFORCE. Since the gradient estimator relies directly on Monte Carlo returns, the learning curve shows considerable variance throughout training. These returns can vary significantly between episodes even for similar policies, leading to noisy updates and unstable learning.

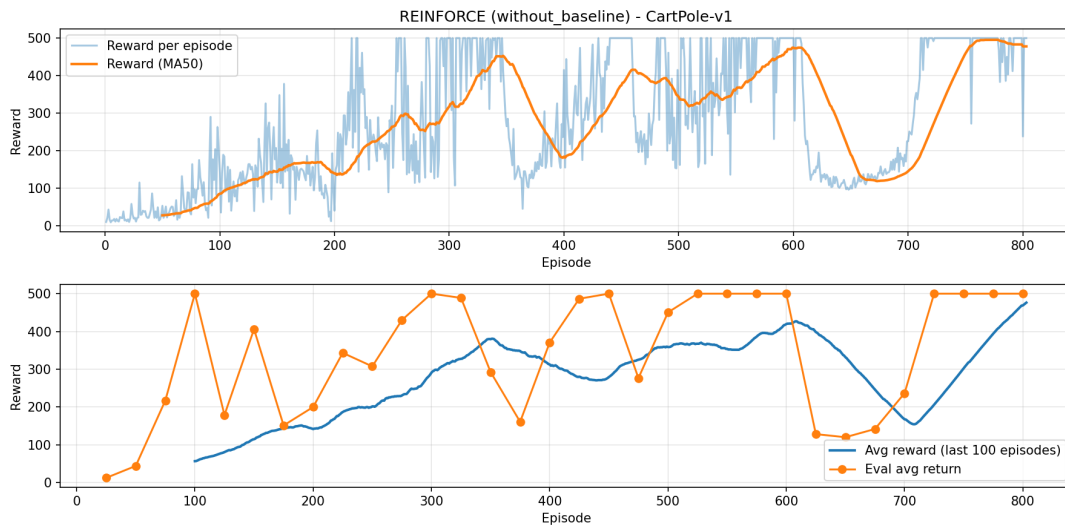


Figure 1: Learning curve for REINFORCE without baseline.

In contrast, Figure 2 demonstrates the behavior when using a baseline. The learning curve is noticeably smoother, with reduced fluctuations and more consistent improvement over time.

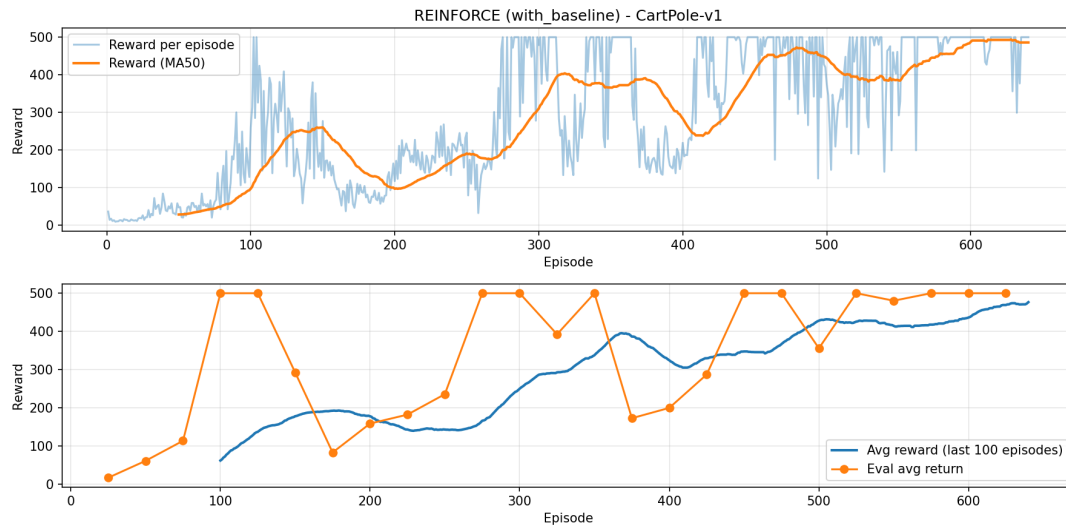


Figure 2: Learning curve for REINFORCE with baseline.

Comparison and Analysis

The performance of both approaches are shown below.

Metric	Without Baseline	With Baseline
Episodes to solve	803	640
Convergence time	Slower	20.3% faster

Table 1: Performance comparison between the REINFORCE algorithms.

As shown in table 1, the baseline version converges in 640 episodes compared to 803 for the basic approach. This speedup comes from the way advantages are computed. Instead of using raw returns to weight the policy gradient, we subtract a state-value estimate. When an action leads to a return that's above average for that state, the advantage is positive and we increase its probability, when it's below average, the advantage is negative and we decrease it, so in fact we're not asking whether an action was good in absolute terms, but whether it was better than expected. Importantly, subtracting the baseline doesn't bias the gradient estimate (as proven in Question 1.2). The expected value of the baseline term is zero when we average over all possible actions, so we're still optimizing the same objective. What changes is the variance of our gradient estimates. Lower variance means each update points more reliably in the direction of improvement, allowing the algorithm to learn faster with fewer episodes.

Section 2 - Advantage Actor-Critic

Question 1: Why is using the TD-error of the value function for the update of the policy network parameters practically the same as using the advantage estimate (of equation 1)? Prove.

In actor-critic algorithm, the advantage is defined as:

$$A_{\pi}(s_t, a_t) = Q_{\pi}(s_t, a_t) - V_{\pi}(s_t)$$

The TD-error (using a value function) is:

$$\delta_{\pi} = r_t + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_t)$$

For a fixed state-action pair (s_t, a_t) , we consider the average outcome of the reward and next state that result from taking action a_t in state s_t :

$$\begin{aligned} E[\delta_{\pi}] &= E[r_t + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_t) \mid s_t, a_t] \\ &= E[r_t + \gamma V_{\pi}(s_{t+1}) \mid s_t, a_t] - V_{\pi}(s_t). \end{aligned}$$

From the definition of the action-value function:

$$Q_{\pi}(s_t, a_t) = E[r_t + \gamma V_{\pi}(s_{t+1}) \mid s_t, a_t].$$

Therefore:

$$E[\delta_{\pi}] = Q_{\pi}(s_t, a_t) - V_{\pi}(s_t) = A_{\pi}(s_t, a_t).$$

So we can say that the TD-error δ_{π} is a one-step unbiased estimator of the advantage. In practice, the value function V_{π} is approximated by a learned critic, and therefore the TD-error is only approximately equal to the true advantage.

Question 2: Explain which function is the actor and which is the critic of the model and what is the role of each one.

In the actor-critic model, the actor is the policy network $\pi_{\theta}(a \mid s)$, and its role is to select actions. Given a state s_t , the actor selects an action according to the current policy and is updated to favor actions that lead to higher expected rewards. The critic is a value estimator, typically the state-value function $V(s)$, and its role is to evaluate how good the current state or action is under the current policy and to provide a learning signal for the

actor. This signal is given by the TD-error:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

which indicates whether the observed outcome was better or worse than expected. Using this feedback, the critic guides the policy updates of the actor and helps reduce the variance of learning.

Actor-Critic Algorithm Implementation

The Actor-Critic algorithm combines policy gradient methods with value function approximation. Unlike REINFORCE, which waits until the end of an episode to compute Monte Carlo returns, Actor-Critic updates the policy at every timestep using temporal difference (TD) error. The critic learns to estimate the value function $V(s)$, while the actor uses the TD error $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ to update the policy. This bootstrapping aiming to reduces variance compared to Monte Carlo methods and enables online learning.

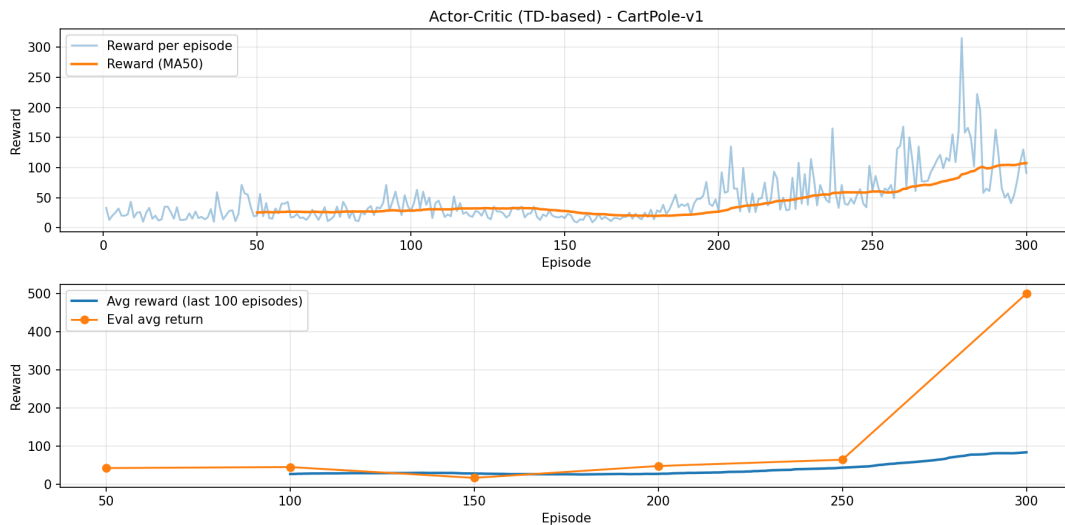


Figure 3: Learning performance of the Actor-Critic algorithm..

Comparison Across Algorithms

Figure 4 compares all three policy gradient approaches side-by-side: REINFORCE without baseline, REINFORCE with baseline, and Actor-Critic. Table 2 summarizes their performance.

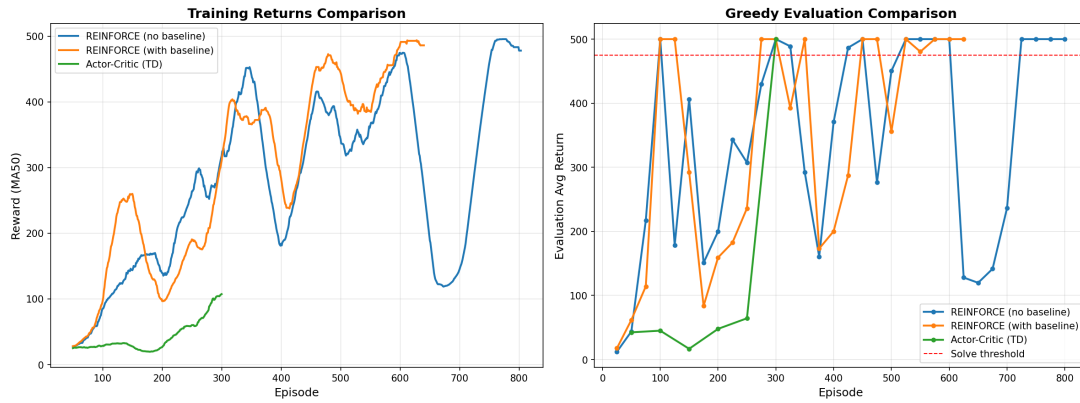


Figure 4: Comparison of learning performance across policy gradient methods.

Metric	REINFORCE (no baseline)	REINFORCE (baseline)	Actor-Critic
Episodes to solve	803	640	300
Update frequency	Per episode	Per episode	Per timestep
Convergence speed	Slower	20.3% faster	63% faster

Table 2: Performance comparison across all three policy gradient methods.

Actor-Critic solved the environment in 300 episodes, compared to 640 episodes for REINFORCE with baseline and 803 for basic REINFORCE. This represents a 53% improvement in convergence speed over the baseline version and a 63% improvement over basic REINFORCE. It's worth noting that Actor-Critic uses a different evaluation criterion, periodically testing the policy in greedy mode (selecting the best short-term action) and tracking the best performance achieved. This evaluation metric reached 500, well above the 475 threshold. This differs from the REINFORCE implementations which track the moving average of training episode returns. The greedy evaluation provides a cleaner measure of the learned policy's quality by removing exploration noise. The key advantage of Actor-Critic lies in when it updates the policy. REINFORCE must complete an entire episode to compute returns $G_t = \sum_{k=0}^T \gamma^k r_{t+k}$, while Actor-Critic updates after every timestep using the TD error $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$. This creates a tradeoff, Monte Carlo returns are unbiased but high variance, they depend on many random future events. Actor-Critic replaces the uncertain future with a value estimate, reducing variance but introducing bias (we're using one estimate to update another). In practice, the variance reduction dominates, allowing faster convergence as the results confirm.

Appendix

Code Execution Explanation

To run the submitted scripts, a `conda` environment file (`drl_hw2_env.yaml`) is provided. Create and activate the environment by running the following commands from the same folder the `yaml` file is located:

```
1 conda env create -f drl_hw2_env.yaml
2 conda activate drl_hw2_env
```

After activating the environment, navigate to the folder containing the Python files and run the wanted script:

- `reinforce.py` runs section 1.
- `actor_critic.py` runs section 2.